Université de Limoges — Faculté des Sciences & Techniques

Faculté des Sciences et Techniques de Limoges

L2 - Bases du langage C
TP2 - Manipulation de bits

Maxime MARIA

Respectez bien les consignes et testez le bon fonctionnement de tous vos programmes ! Pensez aussi à sauvegarder régulièrement ainsi qu'à garder tous vos codes !

1 Opérateurs logiques

L'objectif de cet exercice est simplement de vous faire utiliser les opérateurs logiques (ou booléens).
Dans le calendrier grégorien, une année est bissextile si l'une des deux conditions suivantes est respectée :

- l'année est divisible par 4 mais pas par 100 ;
- l'année est divisible par 400.

1. Créez un programme qui compte et affiche les années bissextiles entre deux bornes passées en paramètres. La Figure 1 montre l'affichage attendu pour une exécution avec 1990 et 2024 en arguments.

```
Les annees bissextiles entre 1990 et 2024 sont :
- 1992
- 1996
- 2000
- 2004
- 2008
- 2012
- 2016
- 2020
- 2024
Il y en a 9.
```

Figure 1 – Affichage attendu pour l'exercice 1.

2 Opérateurs bit à bit

1. Dans le premier TP, vous avez affiché la représentation binaire d'un nombre. Une façon de faire est d'utiliser l'opérateur *modulo* (%) et la division par 2. Dans un nouveau programme, réécrivez cette fonction en remplaçant les opérations arithmétiques par des opérations bit à bit.
   Indice : que se passe-t-il quand je décale mes bits vers la gauche ? Vers la droite ? Comment vérifier la parité de mon nombre ?
2. Écrivez une fonction qui, à partir d'un entier non signé de 32 bits passé en paramètre, retourne un nouvel entier dont les deux moitiés ont été permutées. **Exemple :** si l'entier 00000000 00110011 01001001 11111110 est passé en paramètre, alors la fonction retourne 01001001 11111110 00000000 00110011.

Pour les questions suivantes, on utilise l'ordre suivant pour les bits : $0_n 0_{n-1} ... 0_{i+1} 0_i 0_{i-1} ... 0_1 0_0$.

3. Écrivez une fonction qui retourne la valeur du bit à la position i d'un entier non signé de 32 bits passé en paramètre.
4. Écrivez une fonction qui, à partir d'un entier non signé de 32 bits passé en paramètre, retourne un nouvel entier dont le bit à la position i a été inversé (0 devient 1 et 1 devient 0).
5. Écrivez une fonction qui prend trois paramètres : la position i d'un bit, un entier non signé de 32 bits et un bit b. Elle renvoie l'entier non signé avec le bit à la position i fixé à b. Utilisez les deux fonctions précédemment codées !

3 Le retour de la chèvre

Dans cet exercice, nous allons manipuler les pixels d'une image en utilisant les opérateurs bit à bit et les masques.



(a) Image source (b) Inversion des couleurs (c) Filtre sépia

FIGURE 2 – Illustrations de l'exercice 3.

Téléchargez le dossier `TP2_images` depuis la plateforme pédagogique. Il contient :

- un dossier `stb` contenant des fichiers de la bibliothèque *stb*[1] pour charger et sauvegarder une image ;
- deux fichiers `image.h` et `image.c` qui contiennent les fonctions nécessaires pour manipuler facilement les images pour cet exercice avec les connaissances que vous avez pour le moment, vous n'êtes pas sensés comprendre le code qu'ils contiennent ;
- un fichier `main.c` qui contient le programme principal et montre comment utiliser les fonctions inclues via `#include"image.h"` ;
- la fameuse image `chevre.jpg`.

Rappel (ou pas) : les couleurs sont définies par un mélange de trois couleurs de base : rouge, vert et bleu. On parle alors de couleur RGB (*Red*, *Green*, *Blue*). Chacune de ces couleurs de base (appelées composantes) est définie sur l'intervalle $[0; 255]$ donc codée sur 8 bits. L'opacité des couleurs est représentée par une quatrième composante nommée « alpha », également définie sur $[0; 255]$ (0 = totalement transparent ; 255 = totalement opaque). Une couleur RGBA est donc codée sur 32 bits.
Dans cet exercice, les couleurs sont stockées sous la forme d'entiers non signés sur 32 bits (`uint32_t` de `<stdint.h>`) tels que : `AAAAAAAABBBBBBBBGGGGGGGGRRRRRRRR`. Par exemple, du rouge totalement opaque est codé tel que : `11111111000000000000000011111111` ; en hexadécimal : `FF0000FF`.
Pour écrire une valeur avec la notation hexadécimale en C, il faut écrire `0x` avant le code hexadécimal, *e.g.* `uint32_t white = 0xFFFFFFFF;`

1. Dans un premier temps, vous allez seulement copier l'image d'entrée `img` dans `imgCopy`. Pour cela vous devrez parcourir l'ensemble des pixels de l'image et utiliser les fonctions `getPixelColor` et `setPixelColor` de `"image.h"`. Écrivez votre code au niveau du `// CODE HERE`. Vérifier le fonctionnement de la copie en regardant le fichier `resultCopy.png` dans le dossier de votre programme (*cf.* Figure 2(a)).
   N.B. : pour compiler, vous allez devoir utiliser l'option `-lm` pour pouvoir utiliser la bibliothèque de maths.
2. Nous allons maintenant inverser (composante par composante) les couleurs de notre image afin d'obtenir l'image 2(b) : le blanc devient noir, le noir devient blanc, le jaune devient bleu, *etc.* Le tableau ci-dessous illustre cette inversion pour une couleur (R=121, G=103, B=227, A=255). Attention, la composante alpha ne doit pas changer !

	R			G			B			A		
	Bin	Dec	Hex	Bin	Dec	Hex	Bin	Dec	Hex	Bin	Dec	Hex
In	0 1 1 1 1 0 0 1	121	79	0 1 1 0 0 1 1 1	103	67	1 1 1 0 0 0 1 1	227	E3	1 1 1 1 1 1 1 1	255	FF
Out	1 0 0 0 0 1 1 0	134	86	1 0 0 1 1 0 0 0	152	98	0 0 0 1 1 1 0 0	28	1C	1 1 1 1 1 1 1 1	255	FF

   - Créez une bibliothèque locale `"color_process"` et ajoutez-y une fonction `invertColor` qui, à partir d'une couleur d'origine (`inColor`) passée en arguments, retourne l'inverse de la couleur. Vous n'utiliserez que des opérateurs bit à bit.

1. https://github.com/nothings/stb

- Utilisez votre fonction pour remplir les pixels de l'image `imgInvert`.

3. *(Optionnel)* Nous voulons maintenant appliquer un filtre « sépia » à notre image afin d'obtenir l'image 2(c). Pour cela vous devez modifier les composantes RGB de chaque pixel tel que :

$$\begin{cases} outR = min(255, (0.393 \times inR + 0.769 \times inG + 0.189 \times inB)) \\ outG = min(255, (0.349 \times inR + 0.686 \times inG + 0.168 \times inB)) \\ outB = min(255, (0.272 \times inR + 0.534 \times inG + 0.131 \times inB)) \end{cases}$$

- Ajoutez à votre bibliothèque une fonction `colorToSepia` qui, à partir d'une couleur d'origine (`inColor`) passée en arguments, applique le filtre sépia et retourne la couleur résultante.
- Utilisez votre fonction pour remplir les pixels de l'image `imgSepia`.